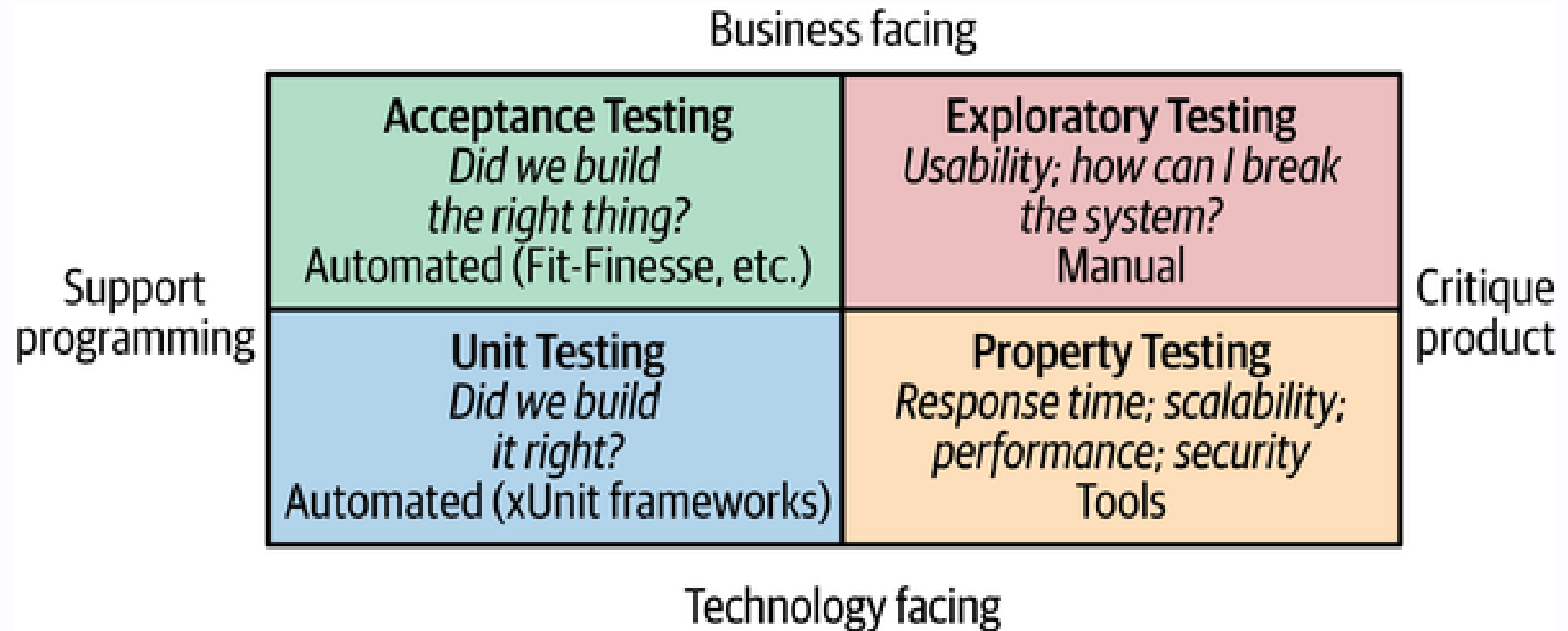


Unidad 7

Control de Calidad

En el desarrollo ágil, en particular en microservicios, nos enfrentamos a la contradicción de poner en producción nuestro software lo más rápido posible y asegurar que cumpla estándares de calidad.



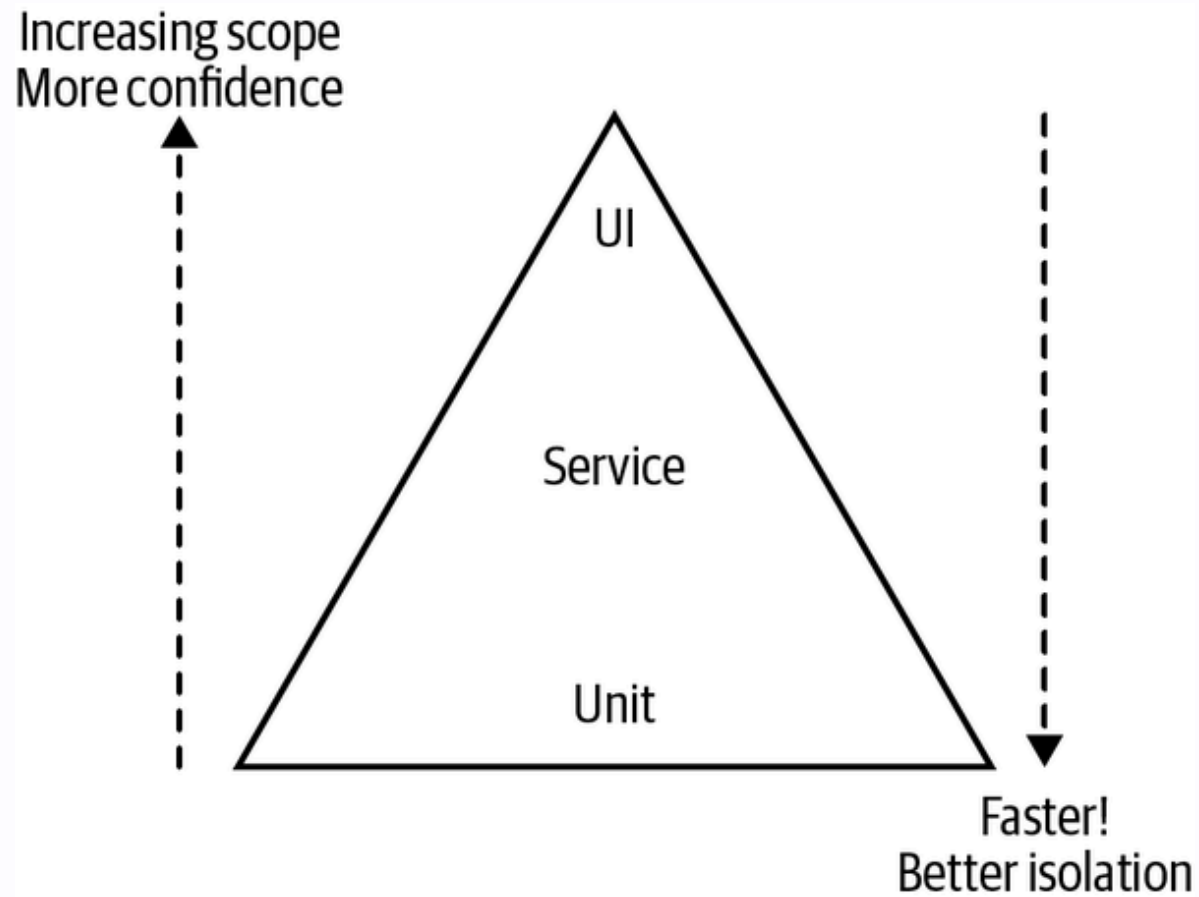
Brian Marick's testing quadrant.

- La mayoría de estas pruebas se centran en asegurar la calidad previo a desplegar la aplicación en producción.
- Que estas pruebas pasen (o fallen) decide si el sistema debe ser desplegado.
- Un porcentaje importante de estas pruebas deben ser automáticas (y su ejecución también).
- En este capítulo vamos a ignorar el control de calidad manual, no porque no sea importante (¡lo es!). Este no es un curso de Control de Calidad.



Alcance de las pruebas

Se pueden dividir las pruebas automatizadas en pruebas unitarias (*unit tests*), pruebas de servicios (*service tests*) y de interfaz.



Mike Cohn's test pyramid.

Más arriba en la pirámide, nuestra confianza en las pruebas aumenta.

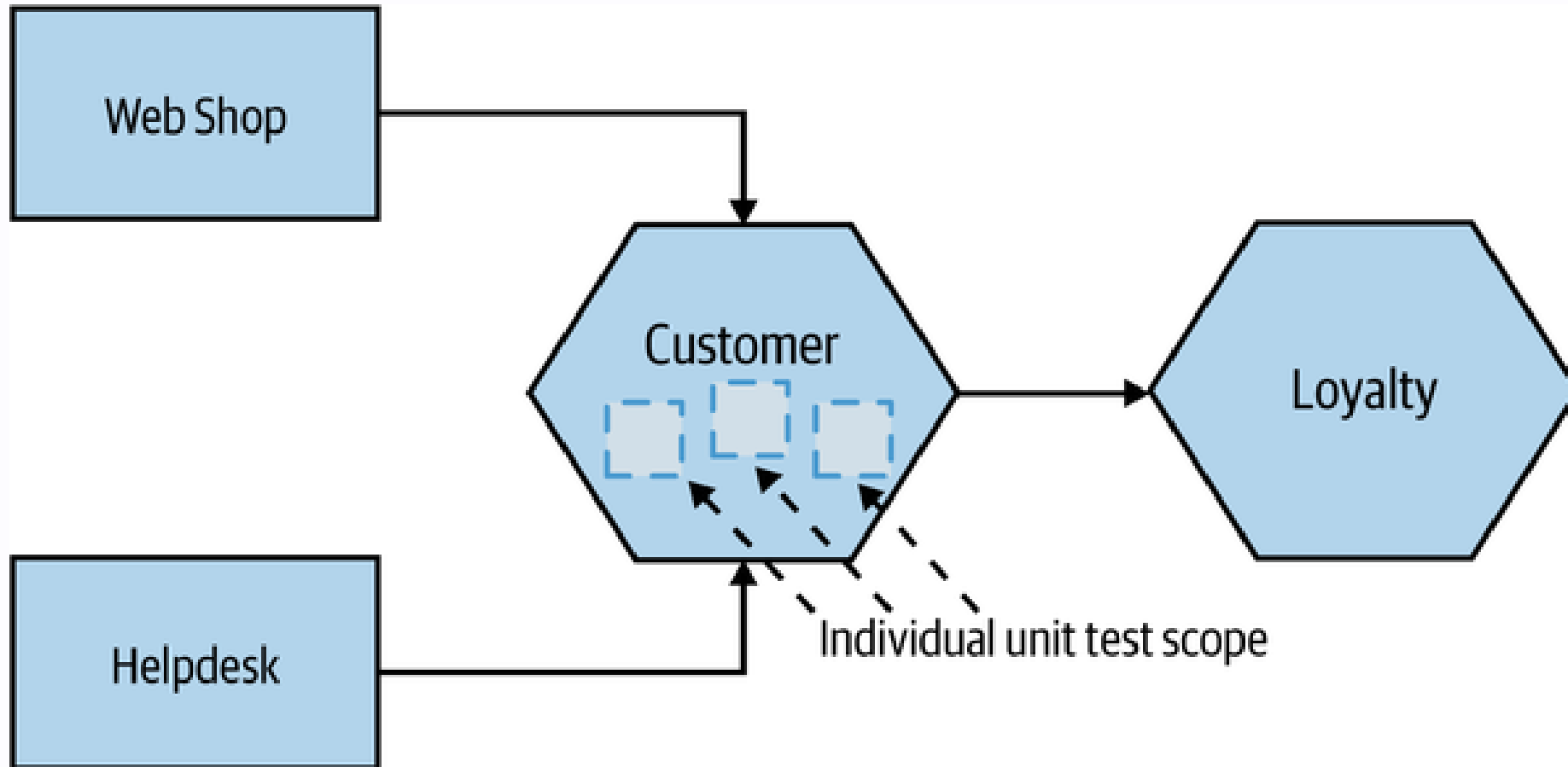
Más abajo en la pirámide, más rápido es identificar los errores, el ciclo de retroalimentación es más corto y el error está más aislado.



Pruebas Unitarias

Prueban, habitualmente, una sola función o método. Se popularizaron por metodologías como test-driven development (TDD). Con estas pruebas se espera capturar la mayoría de los errores.

El objetivo principal de estas pruebas es tener retroalimentación rápida de si la funcionalidad está bien implementada.



Ejemplo de pruebas unitarias

```
# main.py
def mean(values):
    return sum(values) / len(values)

# tests.py
import unittest
from main import mean

class TestMean(unittest.TestCase):
    def test_promedio(self):
        self.assertEqual(mean([10, 10, 10]), 10)

    def test_lista_vacia(self):
        # Caso borde: aquí es donde un test aporta valor.
        # mean([]) divide por cero.
        with self.assertRaises(ZeroDivisionError):
            mean([])

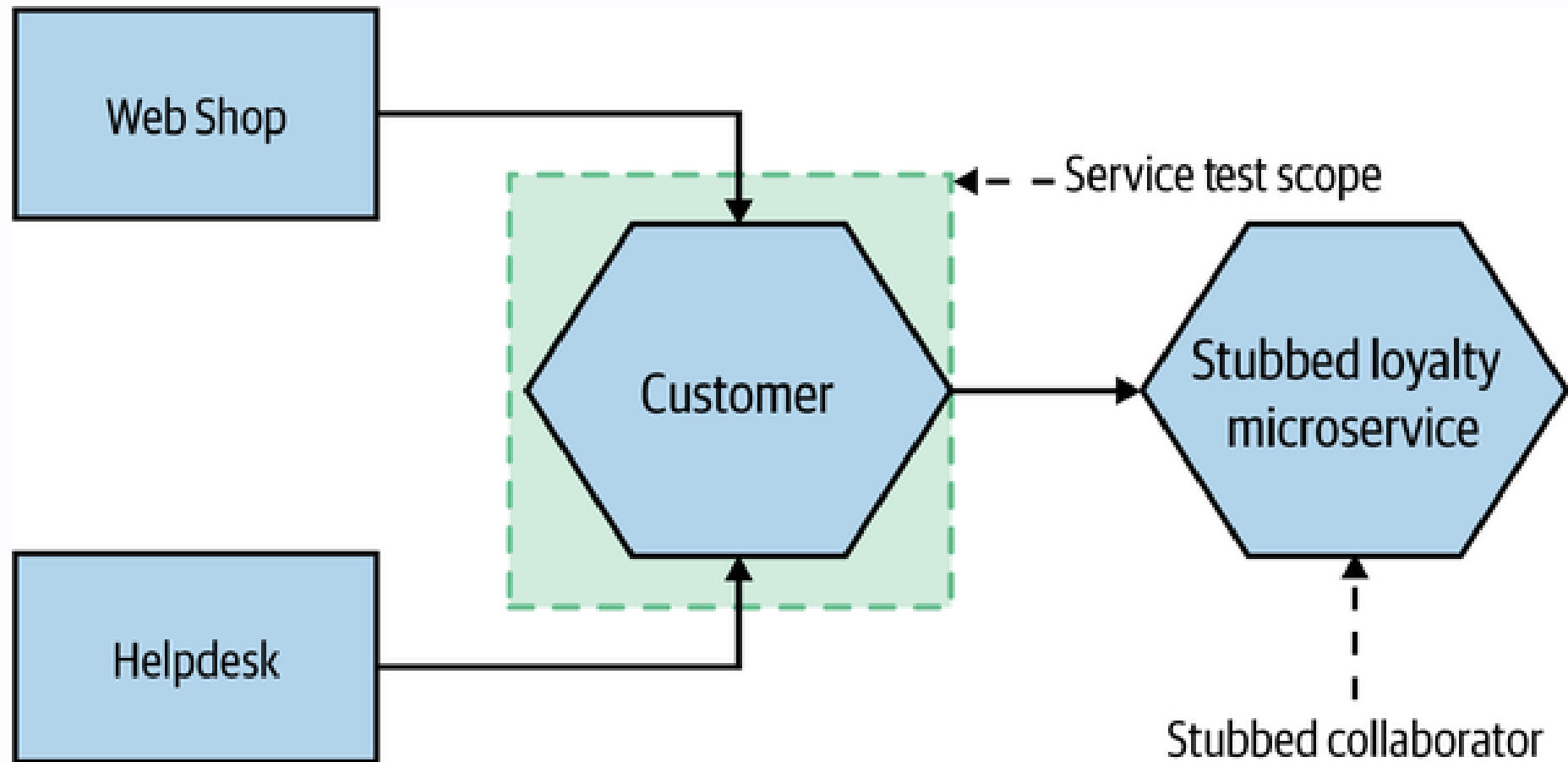
if __name__ == '__main__':
    unittest.main()
```



Pruebas de servicios

Son pruebas directamente a los microservicios. En una aplicación monolítica, se prueban los recursos que provee el servicio a la interfaz.

En microservicios, se prueba cada uno de los servicios.



🧩 Ejemplo de pruebas de servicios

```
from fastapi.testclient import TestClient

from .main import app

client = TestClient(app)

def test_read_item():
    response = client.get("/items/foo",
                           headers={"X-Token": "coneofsilence"})
    assert response.status_code == 200
    assert response.json() == {"id": "foo",
                               "title": "Foo",
                               "description": "There goes my hero"}
```

...

```
def test_read_item_bad_token():
    response = client.get("/items/foo",
                          headers={"X-Token": "hailhydra"})
    assert response.status_code == 400
    assert response.json() == {"detail": "Invalid X-Token header"}

def test_read_inexistent_item():
    response = client.get("/items/baz",
                          headers={"X-Token": "coneofsilence"})
    assert response.status_code == 404
    assert response.json() == {"detail": "Item not found"}
```

- Al realizarse pruebas con componentes en una red o en bases de datos, habitualmente son más lentas que las pruebas unitarias.
- Cuando se requiere probar datos que requieren interacción con otros servicios existen distintas estrategias para aislar las pruebas.

÷ Estrategias para aislar las pruebas

- **Objetos ficticios** (*dummy*) que se pasan pero nunca se usan. Por lo general, solo se usan para llenar las listas de parámetros.
- **Objetos falsos** (*fake*) que son similares a valores reales pero con alguna clase de atajo (marcados como *fake* o se guardan en una base de datos en memoria por ejemplo).

- Los ***stubs*** proporcionan respuestas fijas a las llamadas realizadas durante la prueba. Con un *stub* verificamos el **estado** resultante: le damos una respuesta enlatada al código y revisamos que el resultado final sea el esperado.
- Los **simulacros** (*mocks*) registran las interacciones que reciben. Con un *mock* verificamos el **comportamiento**: qué se llamó, cuántas veces y con qué argumentos.

🧩 Ejemplo de *mock* con `unittest.mock`

```
from unittest.mock import Mock

def notificar_pago(usuario, notificador):
    notificador.enviar(usuario, "Pago recibido")

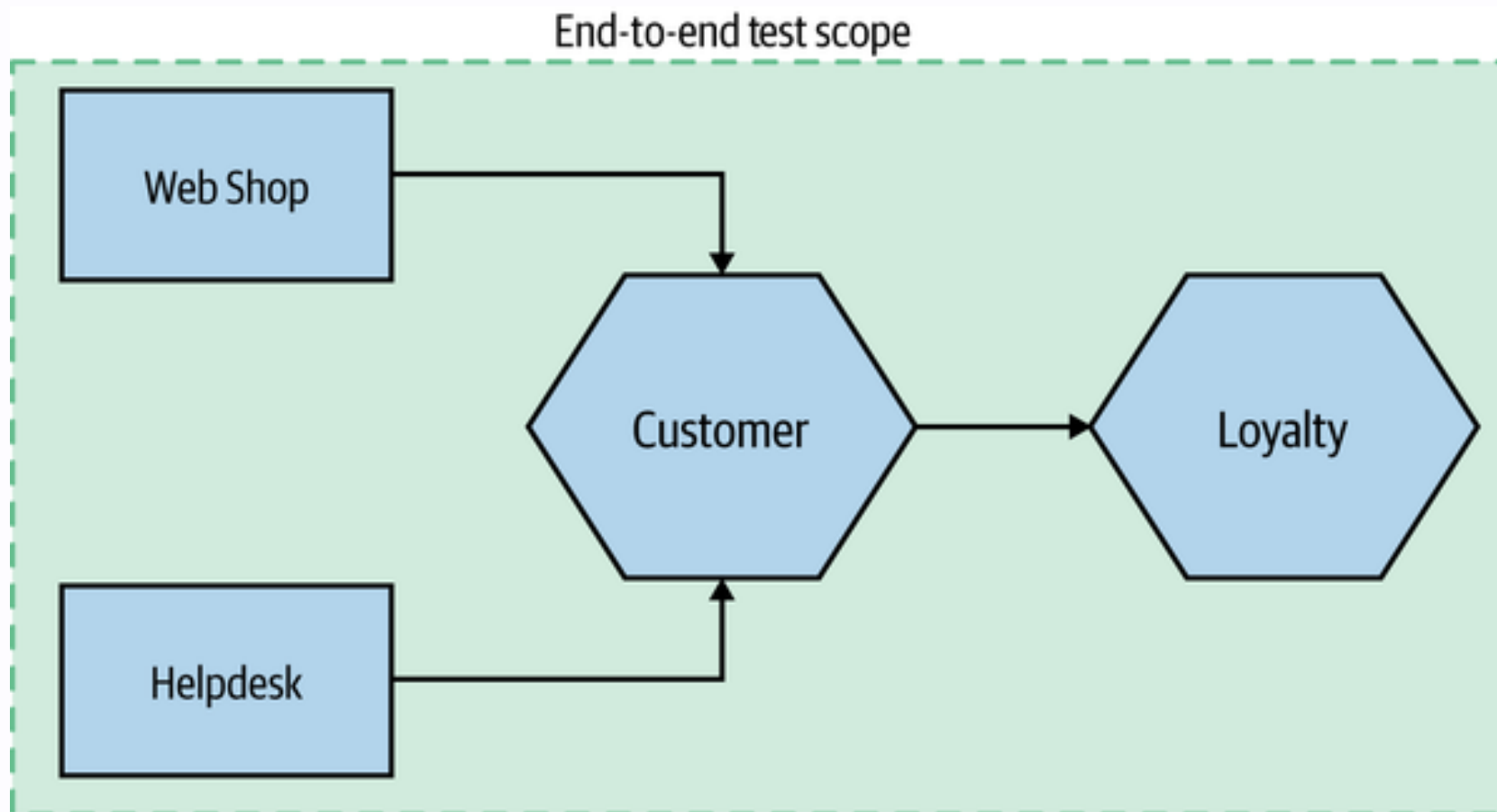
def test_notifica_al_usuario():
    notificador = Mock()
    notificar_pago("ana", notificador)
    # No verificamos un resultado: verificamos
    # que la llamada ocurrió y con qué argumentos.
    notificador.enviar.assert_called_with("ana", "Pago recibido")
```



Pruebas de principio a fin

Habitualmente se realizan sobre la interfaz aunque se pueden realizar probando interacciones complejas entre varios servicios.

Que estas pruebas funcionen da un alto nivel de confianza en que el conjunto de funcionalidades a probar funciona.



Para estas pruebas habitualmente se utilizan frameworks como [Selenium](#). Se debe automatizar la ejecución de estas pruebas.

Un problema de estas pruebas es que al fallar, es difícil determinar la causa de la falla. Sobre todo si hay muchos servicios relacionados.

Además, al fallar una prueba que involucra múltiples servicios ¿De qué equipo es la responsabilidad?



Pruebas de contrato

Un **contrato** es el acuerdo entre un servicio consumidor y uno proveedor: qué peticiones va a hacer el consumidor y qué respuestas espera (rutas, campos, tipos, códigos de estado).

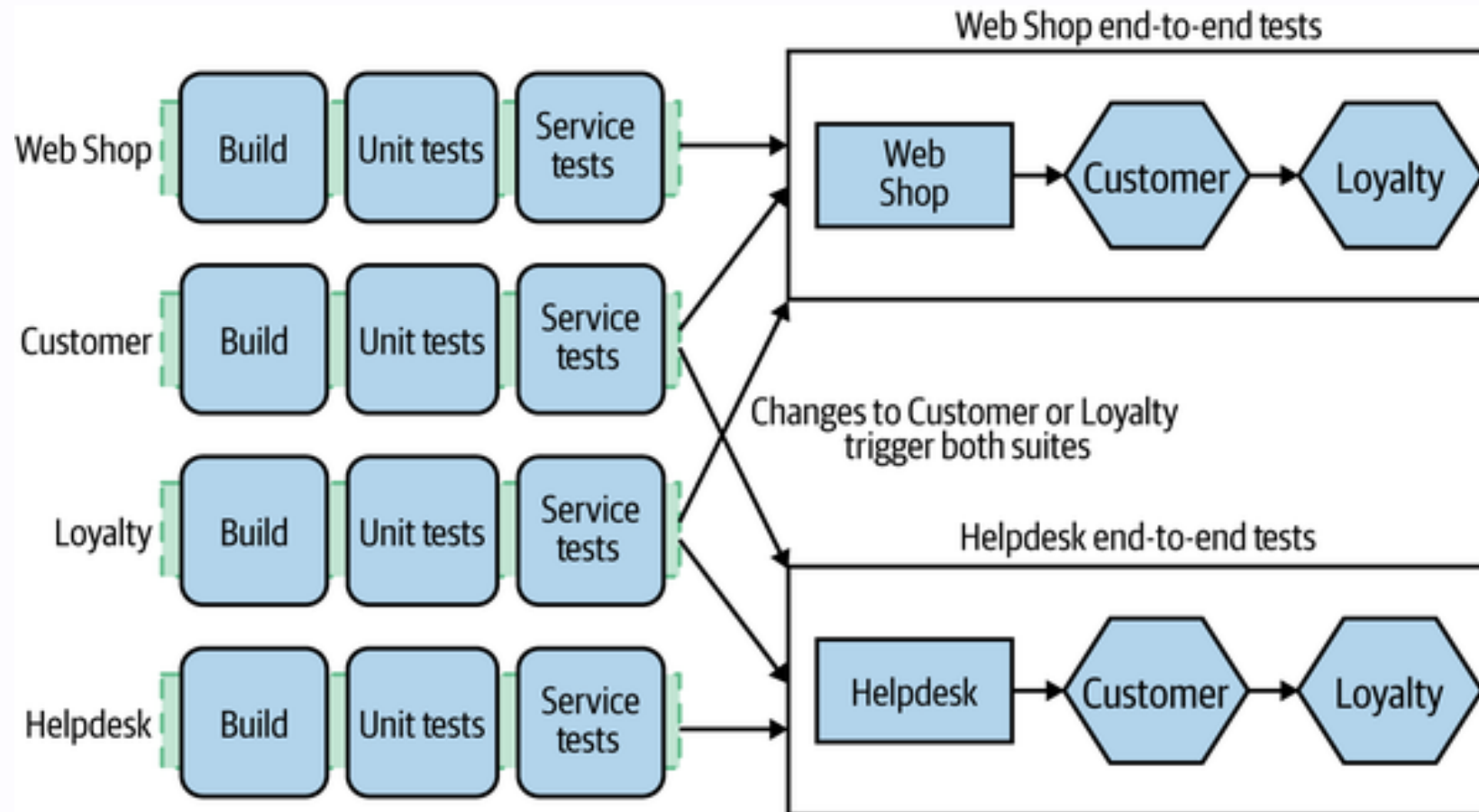
Las pruebas de contrato verifican ese acuerdo sin levantar ambos servicios juntos.

En los *consumer-driven contracts* (con herramientas como **Pact**) el flujo es:

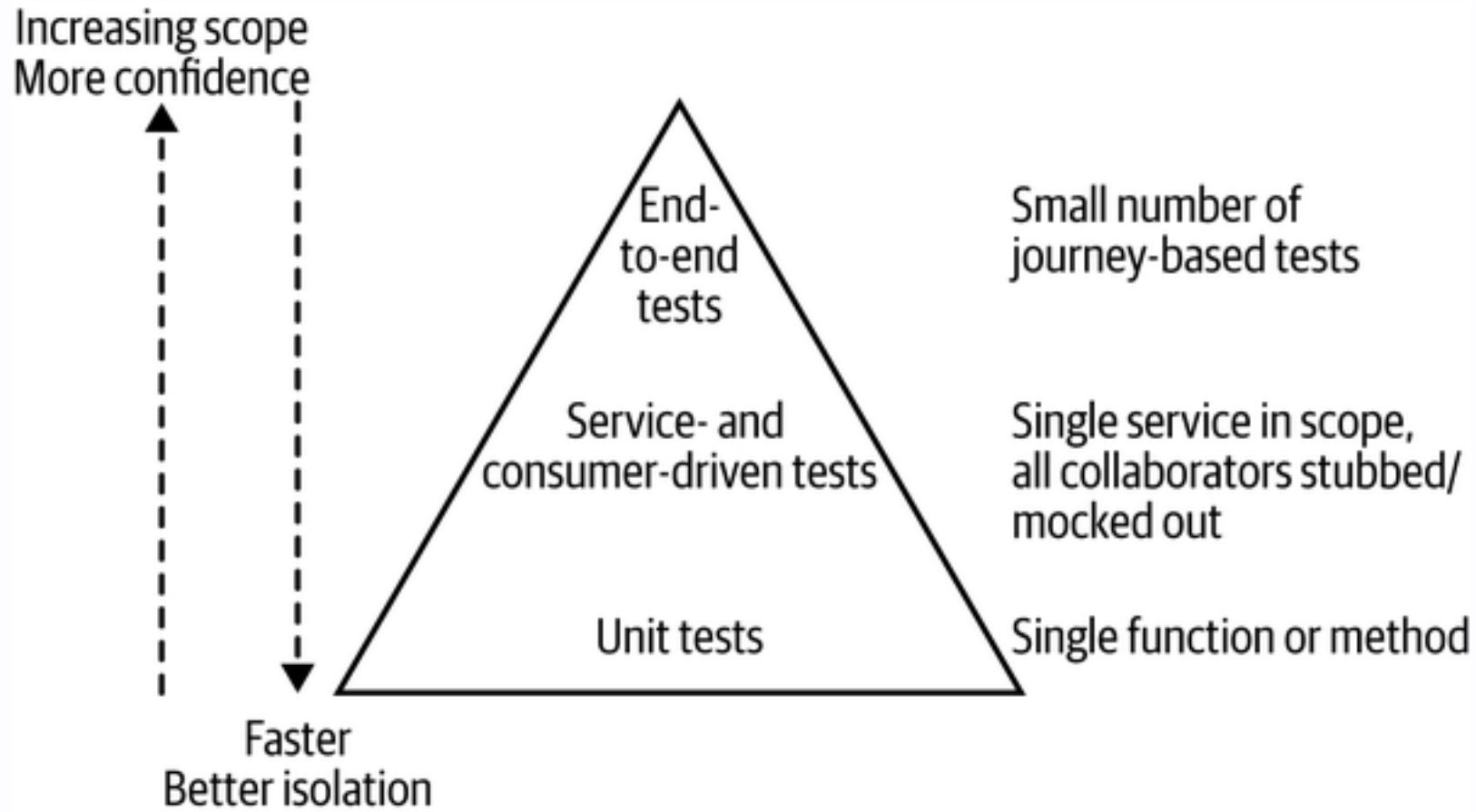
1. El equipo consumidor escribe pruebas contra un *mock* del proveedor. De ahí se genera el **pacto** (un archivo con las interacciones esperadas).
2. El pacto se comparte con el equipo proveedor.
3. El CI del proveedor ejecuta el pacto contra el servicio real: si una respuesta deja de cumplir lo que el consumidor espera, el pipeline falla **antes** de desplegar.

¿Por qué importan? Porque responden la pregunta de la responsabilidad: cada pacto tiene un consumidor y un proveedor claros. Si el pacto falla, se sabe qué equipo rompió el acuerdo.

Con contratos verificados en CI, gran parte de las pruebas de principio a fin entre servicios se vuelve innecesaria: la integración ya está cubierta, servicio por servicio, sin el costo ni la fragilidad de levantar todo el sistema.



En resumen...



Pruebas en producción

Los sistemas en producción se enfrentan a escenarios distintos a los de entornos de pruebas. Gran cantidad de consultas, flujos no probados por parte de los usuarios finales o eventos inesperados. Para esto, habitualmente se implementan estrategias como *health checks* y *canary release*.

Un *health check* es realizar consultas periódicas para corroborar que el servicio está sano (respondiendo normalmente consultas).

Las *canary release* son versiones que utilizan un porcentaje menor de usuarios con la finalidad de que errores descubiertos en flujos reales no lleguen a todos los usuarios.

Ejemplo de *health check* en Kubernetes

```
livenessProbe:
  httpGet:
    # Este end-point debe ser
    # implementado en el servicio
    path: /healthz
    port: 8080
    httpHeaders:
      - name: x-header
        value: Awesome
```

Kubernetes distingue dos tipos de *probe*:

- **livenessProbe** : si falla, Kubernetes **reinicia** el contenedor. Sirve para recuperarse de un servicio colgado.
- **readinessProbe** : si falla, Kubernetes **deja de enviarle tráfico** al pod, sin reiniciarlo.

En microservicios la más importante es la de *readiness*: un servicio que todavía no conecta a su base de datos o no cargó su configuración no debe recibir peticiones, aunque el proceso esté vivo.

Ejemplo de readinessProbe

```
readinessProbe:  
  httpGet:  
    # Debe responder 200 solo cuando el  
    # servicio puede atender peticiones  
    # (por ejemplo, con la BD conectada)  
    path: /ready  
    port: 8080  
  initialDelaySeconds: 5  
  periodSeconds: 10
```



Tarea

Agrega a tu microservicio pruebas de servicio. Se exige:

- Como mínimo, dos pruebas por *end-point*, y al menos una de ellas debe ser un caso de error (entrada inválida, recurso inexistente, credencial incorrecta).
- Las pruebas deben estar aisladas de la base de datos real: usa una BD de pruebas mediante *override* de dependencias de FastAPI o una *fixture* que la cree y destruya.

Rúbrica de la tarea

Criterio	Puntaje
Dos pruebas por end-point, incluyendo un caso de error	40
Aislamiento de la BD (override o fixture)	30
Pruebas corriendo en CI	20
Código de pruebas legible y ordenado	10



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulo 7.
- [FastAPI testing tutorial](#).
- [Mocks Aren't Stubs](#), Martin Fowler (2007).
- [Pact documentation](#)
- [Configure Liveness, Readiness and Startup Probes, Kubernetes](#)